

FBV · 3 CHAPTERS

Django CRUD Complete Code Book

Chapter 1 — CRUD **with forms.py**

Chapter 2 — CRUD **without forms.py** (manual request.POST)

Chapter 3 — Sharing one HTML page for Create + Update

- ✓ Normal CRUD · OneToOne · One-to-Many · ManyToMany
- ✓ Every section fully self-contained: models, views, urls, HTML
- ✓ Function-Based Views only
- ✓ Scenario based: a Book has one Owner, an Owner has many Books

CHAPTER 1

CRUD with forms.py

Every section below is complete on its own: models.py, forms.py, views.py, urls.py and ALL HTML templates are repeated for each case — nothing is shared between sections.

1.1 · Normal CRUD (single model)

Scenario: a simple **Book** table — name and pages. No relationships.

models.py

```
# books/models.py
from django.db import models

class Book(models.Model):
    name = models.CharField(max_length=200)
    pages = models.IntegerField()

    def __str__(self):
        return self.name
```

forms.py

```
# books/forms.py
from django import forms
from .models import Book

class BookForm(forms.ModelForm):
    class Meta:
        model = Book
        fields = ['name', 'pages']
```

views.py

```
# books/views.py
from django.shortcuts import render, redirect, get_object_or_404
from .models import Book
from .forms import BookForm

def book_list(request):
    # READ (all)
    books = Book.objects.all()
    return render(request, 'books/book_list.html', {'object_list': books})

def book_detail(request, pk):
    # READ (one)
    book = get_object_or_404(Book, pk=pk)
    return render(request, 'books/book_detail.html', {'object': book})

def book_create(request):
    # CREATE
    form = BookForm(request.POST or None)
    if form.is_valid():
        form.save()
        return redirect('book_list')
    return render(request, 'books/book_form.html', {'form': form})

def book_update(request, pk):
    # UPDATE
    book = get_object_or_404(Book, pk=pk)
    form = BookForm(request.POST or None, instance=book)
    if form.is_valid():
        form.save()
        return redirect('book_list')
    return render(request, 'books/book_form.html', {'form': form})

def book_delete(request, pk):
    # DELETE
```

```

book = get_object_or_404(Book, pk=pk)
if request.method == 'POST':
    book.delete()
    return redirect('book_list')
return render(request, 'books/book_confirm_delete.html', {'object': book})

```

urls.py

```

# books/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('', views.book_list, name='book_list'),
    path('<int:pk>/', views.book_detail, name='book_detail'),
    path('new/', views.book_create, name='book_new'),
    path('<int:pk>/edit/', views.book_update, name='book_edit'),
    path('<int:pk>/delete/', views.book_delete, name='book_delete'),
]

```

book_list.html

```

{# books/templates/books/book_list.html #}
<h1>Books</h1>
<table border="1">
  <tr><th>Name</th><th>Pages</th><th colspan="3">Actions</th></tr>
  {% for book in object_list %}
  <tr>
    <td>{{ book.name }}</td>
    <td>{{ book.pages }}</td>
    <td><a href="{% url 'book_detail' book.pk %}">view</a></td>
    <td><a href="{% url 'book_edit' book.pk %}">edit</a></td>
    <td><a href="{% url 'book_delete' book.pk %}">delete</a></td>
  </tr>
  {% endfor %}
</table>
<a href="{% url 'book_new' %}">New book</a>

```

book_detail.html

```

{# books/templates/books/book_detail.html #}
<h1>{{ object.name }}</h1>
<p>Pages: {{ object.pages }}</p>
<a href="{% url 'book_list' %}">Back</a>

```

book_form.html

```

{# books/templates/books/book_form.html #}
<h1>Book Form</h1>
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <input type="submit" value="Save">
</form>

```

book_confirm_delete.html

```

{# books/templates/books/book_confirm_delete.html #}
<h1>Delete Book</h1>

```

```
<form method="post">
  {% csrf_token %}
  Are you sure you want to delete "{{ object }}" ?
  <input type="submit" value="Yes, delete">
</form>
```

★ EXAM TIP

This 5-view pattern is the base of everything. The next three sections are the SAME pattern — only the model and the form fields change.

1.2 · OneToOne CRUD

Scenario: each **Book** has exactly ONE **BookProfile** (ISBN + summary), and each profile belongs to exactly one book.

models.py

```
# books/models.py
from django.db import models

class Book(models.Model):
    name = models.CharField(max_length=200)
    pages = models.IntegerField()

    def __str__(self):
        return self.name

class BookProfile(models.Model):
    book = models.OneToOneField(Book, on_delete=models.CASCADE,
                                related_name='profile')
    isbn = models.CharField(max_length=13)
    summary = models.TextField(blank=True)

    def __str__(self):
        return f"Profile of {self.book.name}"
```

forms.py

```
# books/forms.py
from django import forms
from .models import BookProfile

class BookProfileForm(forms.ModelForm):
    class Meta:
        model = BookProfile
        fields = ['book', 'isbn', 'summary']  # 'book' renders as a dropdown
```

views.py

```
# books/views.py
from django.shortcuts import render, redirect, get_object_or_404
from .models import BookProfile
from .forms import BookProfileForm

def profile_list(request):
    # READ (all)
    profiles = BookProfile.objects.select_related('book')
    return render(request, 'books/profile_list.html', {'object_list': profiles})

def profile_detail(request, pk):
    # READ (one)
    profile = get_object_or_404(BookProfile, pk=pk)
    return render(request, 'books/profile_detail.html', {'object': profile})

def profile_create(request):
    # CREATE
    form = BookProfileForm(request.POST or None)
    if form.is_valid():
        form.save()  # IntegrityError if the book already has a profile
        return redirect('profile_list')
    return render(request, 'books/profile_form.html', {'form': form})
```

```
def profile_update(request, pk):
    # UPDATE
    profile = get_object_or_404(BookProfile, pk=pk)
    form = BookProfileForm(request.POST or None, instance=profile)
    if form.is_valid():
        form.save()
        return redirect('profile_list')
    return render(request, 'books/profile_form.html', {'form': form})

def profile_delete(request, pk):
    # DELETE
    profile = get_object_or_404(BookProfile, pk=pk)
    if request.method == 'POST':
        profile.delete()
        return redirect('profile_list')
    return render(request, 'books/profile_confirm_delete.html',
                  {'object': profile})
```

urls.py

```
# books/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('profiles/', views.profile_list, name='profile_list'),
    path('profiles/<int:pk>/', views.profile_detail, name='profile_detail'),
    path('profiles/new/', views.profile_create, name='profile_new'),
    path('profiles/<int:pk>/edit/', views.profile_update, name='profile_edit'),
    path('profiles/<int:pk>/delete/', views.profile_delete, name='profile_delete'),
]
```

profile_list.html

```
{# books/templates/books/profile_list.html #}
<h1>Book Profiles</h1>
<table border="1">
  <tr><th>Book</th><th>ISBN</th><th colspan="3">Actions</th></tr>
  {% for p in object_list %}
  <tr>
    <td>{{ p.book.name }}</td>      {# follow the OneToOne link #}
    <td>{{ p.isbn }}</td>
    <td><a href="{% url 'profile_detail' p.pk %}">view</a></td>
    <td><a href="{% url 'profile_edit' p.pk %}">edit</a></td>
    <td><a href="{% url 'profile_delete' p.pk %}">delete</a></td>
  </tr>
  {% endfor %}
</table>
<a href="{% url 'profile_new' %}">New profile</a>
```

profile_detail.html

```
{# books/templates/books/profile_detail.html #}
<h1>{{ object.book.name }}</h1>
<p>ISBN: {{ object.isbn }}</p>
<p>Summary: {{ object.summary }}</p>
<a href="{% url 'profile_list' %}">Back</a>
```

profile_form.html

```
{# books/templates/books/profile_form.html #}
<h1>Profile Form</h1>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <input type="submit" value="Save">
</form>
```

profile_confirm_delete.html

```
{# books/templates/books/profile_confirm_delete.html #}
<h1>Delete Profile</h1>
<form method="post">
    {% csrf_token %}
    Are you sure you want to delete "{{ object }}" ?
    <input type="submit" value="Yes, delete">
</form>
```

📌 REMEMBER

OneToOne reverse access gives ONE object: `book.profile` (no `.all()`). Creating a second profile for the same book raises `IntegrityError` — the field is a `ForeignKey` + `UNIQUE` under the hood.

1.3 · One-to-Many CRUD (ForeignKey)

Scenario: a **Book** has one **Owner**, an **Owner** has many **Books**. The ForeignKey lives on Book (the "many" side).

models.py

```
# books/models.py
from django.db import models

class Owner(models.Model):
    name = models.CharField(max_length=100)

    def __str__(self):
        return self.name

class Book(models.Model):
    owner = models.ForeignKey(Owner, on_delete=models.CASCADE,
                              related_name='books')
    name = models.CharField(max_length=200)
    pages = models.IntegerField()

    def __str__(self):
        return self.name

# book.owner          -> the Owner of a book
# owner.books.all()   -> all Books of an owner
```

forms.py

```
# books/forms.py
from django import forms
from .models import Book

class BookForm(forms.ModelForm):
    class Meta:
        model = Book
        fields = ['owner', 'name', 'pages'] # 'owner' -> dropdown of owners
```

views.py

```
# books/views.py
from django.shortcuts import render, redirect, get_object_or_404
from .models import Book, Owner
from .forms import BookForm

def book_list(request):
    books = Book.objects.select_related('owner') # READ (all)
    # 1 query, not N+1
    return render(request, 'books/book_list.html', {'object_list': books})

def book_detail(request, pk):
    book = get_object_or_404(Book, pk=pk) # READ (one)
    return render(request, 'books/book_detail.html', {'object': book})

def owner_books(request, owner_pk):
    owner = get_object_or_404(Owner, pk=owner_pk) # READ (filtered)
    return render(request, 'books/book_list.html',
                  {'object_list': owner.books.all(), 'owner': owner})
```

```

def book_create(request):
    # CREATE
    form = BookForm(request.POST or None)
    if form.is_valid():
        form.save()
        return redirect('book_list')
    return render(request, 'books/book_form.html', {'form': form})

def book_update(request, pk):
    # UPDATE
    book = get_object_or_404(Book, pk=pk)
    form = BookForm(request.POST or None, instance=book)
    if form.is_valid():
        form.save()
        return redirect('book_list')
    return render(request, 'books/book_form.html', {'form': form})

def book_delete(request, pk):
    # DELETE
    book = get_object_or_404(Book, pk=pk)
    if request.method == 'POST':
        book.delete()
        return redirect('book_list')
    return render(request, 'books/book_confirm_delete.html', {'object': book})

```

urls.py

```

# books/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('', views.book_list, name='book_list'),
    path('<int:pk>/', views.book_detail, name='book_detail'),
    path('new/', views.book_create, name='book_new'),
    path('<int:pk>/edit/', views.book_update, name='book_edit'),
    path('<int:pk>/delete/', views.book_delete, name='book_delete'),
    path('owner/<int:owner_pk>/', views.owner_books, name='owner_books'),
]

```

book_list.html

```

{# books/templates/books/book_list.html #}
<h1>Books{% if owner %} of {{ owner.name }}{% endif %}</h1>
<table border="1">
  <tr><th>Name</th><th>Owner</th><th>Pages</th><th colspan="3">Actions</th></tr>
  {% for book in object_list %}
  <tr>
    <td>{{ book.name }}</td>
    <td><a href="{% url 'owner_books' book.owner.pk %}">
      {{ book.owner.name }}</a></td>      {# follow the FK #}
    <td>{{ book.pages }}</td>
    <td><a href="{% url 'book_detail' book.pk %}">view</a></td>
    <td><a href="{% url 'book_edit' book.pk %}">edit</a></td>
    <td><a href="{% url 'book_delete' book.pk %}">delete</a></td>
  </tr>
  {% endfor %}
</table>
<a href="{% url 'book_new' %}">New book</a>

```

book_detail.html

```
{# books/templates/books/book_detail.html #}
<h1>{{ object.name }}</h1>
<p>Owner: {{ object.owner.name }}</p>
<p>Pages: {{ object.pages }}</p>
<a href="{% url 'book_list' %}">Back</a>
```

book_form.html

```
{# books/templates/books/book_form.html #}
<h1>Book Form</h1>
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}      {# owner appears automatically as a dropdown #}
  <input type="submit" value="Save">
</form>
```

book_confirm_delete.html

```
{# books/templates/books/book_confirm_delete.html #}
<h1>Delete Book</h1>
<form method="post">
  {% csrf_token %}
  Are you sure you want to delete "{{ object }}" ?
  <input type="submit" value="Yes, delete">
</form>
```

⚠ WATCH OUT

Deleting an Owner with `on_delete=CASCADE` also deletes ALL of that owner's books. Use `PROTECT` to block the delete instead.

1.4 · ManyToMany CRUD

Scenario: a **Book** can have many **Categories**, and a **Category** can contain many **Books**. Django manages a hidden junction table.

models.py

```
# books/models.py
from django.db import models

class Category(models.Model):
    name = models.CharField(max_length=50)

    def __str__(self):
        return self.name

class Book(models.Model):
    name = models.CharField(max_length=200)
    pages = models.IntegerField()
    categories = models.ManyToManyField(Category, related_name='books')

    def __str__(self):
        return self.name

# book.categories.all() -> categories of a book
# category.books.all() -> books in a category
```

forms.py

```
# books/forms.py
from django import forms
from .models import Book

class BookForm(forms.ModelForm):
    class Meta:
        model = Book
        fields = ['name', 'pages', 'categories']
        widgets = {
            'categories': forms.CheckboxSelectMultiple, # nicer than multiselect
        }
```

views.py

```
# books/views.py
from django.shortcuts import render, redirect, get_object_or_404
from .models import Book, Category
from .forms import BookForm

def book_list(request): # READ (all)
    books = Book.objects.prefetch_related('categories')
    return render(request, 'books/book_list.html', {'object_list': books})

def book_detail(request, pk): # READ (one)
    book = get_object_or_404(Book, pk=pk)
    return render(request, 'books/book_detail.html', {'object': book})

def category_books(request, cat_pk): # READ (filtered)
    category = get_object_or_404(Category, pk=cat_pk)
```

```

    return render(request, 'books/book_list.html',
                  {'object_list': category.books.all(), 'category': category})

def book_create(request):
    # CREATE
    form = BookForm(request.POST or None)
    if form.is_valid():
        form.save()
        # saves the Book AND its category links
        return redirect('book_list')
    return render(request, 'books/book_form.html', {'form': form})

def book_update(request, pk):
    # UPDATE
    book = get_object_or_404(Book, pk=pk)
    form = BookForm(request.POST or None, instance=book)
    if form.is_valid():
        form.save()
        # replaces the links with the new selection
        return redirect('book_list')
    return render(request, 'books/book_form.html', {'form': form})

def book_delete(request, pk):
    # DELETE
    book = get_object_or_404(Book, pk=pk)
    if request.method == 'POST':
        book.delete()
        # junction rows removed automatically
        return redirect('book_list')
    return render(request, 'books/book_confirm_delete.html', {'object': book})

```

urls.py

```

# books/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('', views.book_list, name='book_list'),
    path('<int:pk>/', views.book_detail, name='book_detail'),
    path('new/', views.book_create, name='book_new'),
    path('<int:pk>/edit/', views.book_update, name='book_edit'),
    path('<int:pk>/delete/', views.book_delete, name='book_delete'),
    path('category/<int:cat_pk>/', views.category_books, name='category_books'),
]

```

book_list.html

```

{# books/templates/books/book_list.html #}
<h1>Books{% if category %} in {{ category.name }}{% endif %}</h1>
<table border="1">
  <tr><th>Name</th><th>Categories</th><th colspan="3">Actions</th></tr>
  {% for book in object_list %}
  <tr>
    <td>{{ book.name }}</td>
    <td>
      {% for cat in book.categories.all %}
      <a href="{% url 'category_books' cat.pk %}">{{ cat.name }}</a>
      {%- if not forloop.last %}, {% endif %}
      {% empty %} (none) {% endfor %}
    </td>
    <td><a href="{% url 'book_detail' book.pk %}">view</a></td>
    <td><a href="{% url 'book_edit' book.pk %}">edit</a></td>
    <td><a href="{% url 'book_delete' book.pk %}">delete</a></td>
  </tr>
  {% endfor %}

```

```
</table>
<a href="{% url 'book_new' %}">New book</a>
```

book_detail.html

```
{# books/templates/books/book_detail.html #}
<h1>{{ object.name }}</h1>
<p>Pages: {{ object.pages }}</p>
<p>Categories:
    {% for cat in object.categories.all %}
        {{ cat.name }}{% if not forloop.last %}, {% endif %}
    {% endfor %}
</p>
<a href="{% url 'book_list' %}">Back</a>
```

book_form.html

```
{# books/templates/books/book_form.html #}
<h1>Book Form</h1>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}      {# categories appear as checkboxes #}
    <input type="submit" value="Save">
</form>
```

book_confirm_delete.html

```
{# books/templates/books/book_confirm_delete.html #}
<h1>Delete Book</h1>
<form method="post">
    {% csrf_token %}
    Are you sure you want to delete "{{ object }}" ?
    <input type="submit" value="Yes, delete">
</form>
```

⚠ WATCH OUT

If you ever use `form.save(commit=False)` with a M2M form, you MUST call `form.save_m2m()` after `obj.save()`, otherwise the category links are silently lost.

CHAPTER 2

CRUD without forms.py

The same four cases, but with NO `ModelForm` at all. The views read **`request.POST`** manually and the HTML contains hand-written `<input>` fields. Each section is again fully self-contained: `models.py`, `views.py`, `urls.py` and all HTML.

2.1 · Normal CRUD — no forms.py

Scenario: same simple **Book** table. Views build/read every field by hand.

models.py

```
# books/models.py (identical to Chapter 1.1 — no change needed)
from django.db import models

class Book(models.Model):
    name = models.CharField(max_length=200)
    pages = models.IntegerField()

    def __str__(self):
        return self.name
```

views.py

```
# books/views.py — NO forms.py, everything manual
from django.shortcuts import render, redirect, get_object_or_404
from .models import Book

def book_list(request):
    # READ (all)
    books = Book.objects.all()
    return render(request, 'books/book_list.html', {'object_list': books})

def book_detail(request, pk):
    # READ (one)
    book = get_object_or_404(Book, pk=pk)
    return render(request, 'books/book_detail.html', {'object': book})

def book_create(request):
    # CREATE
    if request.method == 'POST':
        Book.objects.create(
            name = request.POST.get('name'),
            pages = request.POST.get('pages'),
        )
        return redirect('book_list')
    return render(request, 'books/book_form.html') # empty form page

def book_update(request, pk):
    # UPDATE
    book = get_object_or_404(Book, pk=pk)
    if request.method == 'POST':
        book.name = request.POST.get('name')
        book.pages = request.POST.get('pages')
        book.save()
        return redirect('book_list')
    # pass the object so the template can pre-fill the inputs
    return render(request, 'books/book_form.html', {'book': book})

def book_delete(request, pk):
    # DELETE
    book = get_object_or_404(Book, pk=pk)
    if request.method == 'POST':
        book.delete()
        return redirect('book_list')
    return render(request, 'books/book_confirm_delete.html', {'object': book})
```


urls.py

```
# books/urls.py (identical pattern to Chapter 1.1)
from django.urls import path
from . import views

urlpatterns = [
    path('', views.book_list, name='book_list'),
    path('<int:pk>/', views.book_detail, name='book_detail'),
    path('new/', views.book_create, name='book_new'),
    path('<int:pk>/edit/', views.book_update, name='book_edit'),
    path('<int:pk>/delete/', views.book_delete, name='book_delete'),
]
```

book_form.html

```
{# books/templates/books/book_form.html – manual inputs, works for #}
{# BOTH create (book missing -> empty) and update (values pre-filled) #}
<h1>Book Form</h1>
<form method="post">
    {% csrf_token %}
    <p>Name:
        <input type="text" name="name"
            value="{{ book.name|default_if_none:'' }}"></p>
    <p>Pages:
        <input type="number" name="pages"
            value="{{ book.pages|default_if_none:'' }}"></p>
    <input type="submit" value="Save">
</form>
```

book_list.html · book_detail.html · book_confirm_delete.html

```
{# books/templates/books/book_list.html #}
<h1>Books</h1>
<table border="1">
    <tr><th>Name</th><th>Pages</th><th colspan="3">Actions</th></tr>
    {% for book in object_list %}
    <tr>
        <td>{{ book.name }}</td><td>{{ book.pages }}</td>
        <td><a href="{% url 'book_detail' book.pk %}">view</a></td>
        <td><a href="{% url 'book_edit' book.pk %}">edit</a></td>
        <td><a href="{% url 'book_delete' book.pk %}">delete</a></td>
    </tr>
    {% endfor %}
</table>
<a href="{% url 'book_new' %}">New book</a>

{# books/templates/books/book_detail.html #}
<h1>{{ object.name }}</h1>
<p>Pages: {{ object.pages }}</p>

{# books/templates/books/book_confirm_delete.html #}
<form method="post">{% csrf_token %}
    Delete "{{ object }}" ? <input type="submit" value="Yes, delete">
</form>
```

⚠ **WATCH OUT**

The big difference vs Chapter 1: NO validation. `request.POST.get('pages')` arrives as a string, and if the user types 'abc' you get a crash/DB error. `ModelForm` did the cleaning + type conversion + error messages for free. In the exam, mention this as the main advantage of forms.py.

2.2 · OneToOne CRUD — no forms.py

Scenario: Book ↔ BookProfile again, but the book dropdown is built by hand in the template.

models.py

```
# books/models.py (identical to Chapter 1.2)
from django.db import models

class Book(models.Model):
    name = models.CharField(max_length=200)
    pages = models.IntegerField()

    def __str__(self):
        return self.name

class BookProfile(models.Model):
    book = models.OneToOneField(Book, on_delete=models.CASCADE,
                                related_name='profile')
    isbn = models.CharField(max_length=13)
    summary = models.TextField(blank=True)

    def __str__(self):
        return f"Profile of {self.book.name}"
```

views.py

```
# books/views.py
from django.shortcuts import render, redirect, get_object_or_404
from .models import Book, BookProfile

def profile_list(request):
    # READ (all)
    profiles = BookProfile.objects.select_related('book')
    return render(request, 'books/profile_list.html', {'object_list': profiles})

def profile_create(request):
    # CREATE
    if request.method == 'POST':
        BookProfile.objects.create(
            book_id = request.POST.get('book'), # '<pk>' from the dropdown
            isbn = request.POST.get('isbn'),
            summary = request.POST.get('summary', ''),
        )
        return redirect('profile_list')
    # only offer books that do NOT have a profile yet (one-to-one rule!)
    books = Book.objects.filter(profile__isnull=True)
    return render(request, 'books/profile_form.html', {'books': books})

def profile_update(request, pk):
    # UPDATE
    profile = get_object_or_404(BookProfile, pk=pk)
    if request.method == 'POST':
        profile.book_id = request.POST.get('book')
        profile.isbn = request.POST.get('isbn')
        profile.summary = request.POST.get('summary', '')
        profile.save()
        return redirect('profile_list')
    # offer free books PLUS the currently linked book
    books = Book.objects.filter(profile__isnull=True) |
    Book.objects.filter(pk=profile.book_id)
    return render(request, 'books/profile_form.html',
```

```

        {'books': books, 'profile': profile})

def profile_delete(request, pk):
    # DELETE
    profile = get_object_or_404(BookProfile, pk=pk)
    if request.method == 'POST':
        profile.delete()
        return redirect('profile_list')
    return render(request, 'books/profile_confirm_delete.html',
                  {'object': profile})

```

urls.py

```

# books/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('profiles/', views.profile_list, name='profile_list'),
    path('profiles/new/', views.profile_create, name='profile_new'),
    path('profiles/<int:pk>/edit/', views.profile_update, name='profile_edit'),
    path('profiles/<int:pk>/delete/', views.profile_delete, name='profile_delete'),
]

```

profile_form.html

```

{# books/templates/books/profile_form.html - manual dropdown #}
<h1>Profile Form</h1>
<form method="post">
    {% csrf_token %}
    <p>Book:
        <select name="book">
            {% for b in books %}
                <option value="{{ b.pk }}"
                    {% if profile.book_id == b.pk %}selected{% endif %}>
                    {{ b.name }}
            </option>
            {% endfor %}
        </select></p>
    <p>ISBN:
        <input type="text" name="isbn"
            value="{{ profile.isbn|default_if_none:'' }}"></p>
    <p>Summary:
        <textarea name="summary">{{ profile.summary|default_if_none:'' }}</textarea></p>
    <input type="submit" value="Save">
</form>

```

profile_list.html · profile_confirm_delete.html

```

{# books/templates/books/profile_list.html #}
<h1>Book Profiles</h1>
<table border="1">
    <tr><th>Book</th><th>ISBN</th><th colspan="2">Actions</th></tr>
    {% for p in object_list %}
    <tr>
        <td>{{ p.book.name }}</td><td>{{ p.isbn }}</td>
        <td><a href="{% url 'profile_edit' p.pk %}">edit</a></td>
        <td><a href="{% url 'profile_delete' p.pk %}">delete</a></td>
    </tr>
    {% endfor %}
</table>

```

```
<a href="{% url 'profile_new' %}">New profile</a>

{% books/templates/books/profile_confirm_delete.html %}
<form method="post">{% csrf_token %}
  Delete "{{ object }}" ? <input type="submit" value="Yes, delete">
</form>
```

★ EXAM TIP

Two manual tricks here: (1) setting the FK by id with `book_id=request.POST.get('book')` — no need to fetch the Book object; (2) filtering the dropdown with `profile__isnull=True` so the one-to-one rule can't be broken from the UI.

2.3 · One-to-Many CRUD — no forms.py

Scenario: Owner → many Books. The owner dropdown is hand-written; every owner can be selected many times.

models.py

```
# books/models.py (identical to Chapter 1.3)
from django.db import models

class Owner(models.Model):
    name = models.CharField(max_length=100)

    def __str__(self):
        return self.name

class Book(models.Model):
    owner = models.ForeignKey(Owner, on_delete=models.CASCADE,
                              related_name='books')
    name = models.CharField(max_length=200)
    pages = models.IntegerField()

    def __str__(self):
        return self.name
```

views.py

```
# books/views.py
from django.shortcuts import render, redirect, get_object_or_404
from .models import Book, Owner

def book_list(request):
    # READ (all)
    books = Book.objects.select_related('owner')
    return render(request, 'books/book_list.html', {'object_list': books})

def book_create(request):
    # CREATE
    if request.method == 'POST':
        Book.objects.create(
            owner_id = request.POST.get('owner'),
            name     = request.POST.get('name'),
            pages    = request.POST.get('pages'),
        )
        return redirect('book_list')
    owners = Owner.objects.all() # dropdown data
    return render(request, 'books/book_form.html', {'owners': owners})

def book_update(request, pk):
    # UPDATE
    book = get_object_or_404(Book, pk=pk)
    if request.method == 'POST':
        book.owner_id = request.POST.get('owner')
        book.name     = request.POST.get('name')
        book.pages    = request.POST.get('pages')
        book.save()
        return redirect('book_list')
    owners = Owner.objects.all()
    return render(request, 'books/book_form.html',
                  {'owners': owners, 'book': book})

def book_delete(request, pk):
    # DELETE
```

```

book = get_object_or_404(Book, pk=pk)
if request.method == 'POST':
    book.delete()
    return redirect('book_list')
return render(request, 'books/book_confirm_delete.html', {'object': book})

```

urls.py

```

# books/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('', views.book_list, name='book_list'),
    path('new/', views.book_create, name='book_new'),
    path('<int:pk>/edit/', views.book_update, name='book_edit'),
    path('<int:pk>/delete/', views.book_delete, name='book_delete'),
]

```

book_form.html

```

{# books/templates/books/book_form.html – manual owner dropdown #}
<h1>Book Form</h1>
<form method="post">
    {% csrf_token %}
    <p>Owner:
        <select name="owner">
            {% for o in owners %}
                <option value="{{ o.pk }}"
                    {% if book.owner_id == o.pk %}>selected{% endif %}>
                    {{ o.name }}
            </option>
            {% endfor %}
        </select></p>
    <p>Name:
        <input type="text" name="name"
            value="{{ book.name|default_if_none:'' }}"></p>
    <p>Pages:
        <input type="number" name="pages"
            value="{{ book.pages|default_if_none:'' }}"></p>
    <input type="submit" value="Save">
</form>

```

book_list.html · book_confirm_delete.html

```

{# books/templates/books/book_list.html #}
<h1>Books</h1>
<table border="1">
    <tr><th>Name</th><th>Owner</th><th>Pages</th><th colspan="2">Actions</th></tr>
    {% for book in object_list %}
    <tr>
        <td>{{ book.name }}</td>
        <td>{{ book.owner.name }}</td>
        <td>{{ book.pages }}</td>
        <td><a href="{% url 'book_edit' book.pk %}">edit</a></td>
        <td><a href="{% url 'book_delete' book.pk %}">delete</a></td>
    </tr>
    {% endfor %}
</table>
<a href="{% url 'book_new' %}">New book</a>

```

```
{# books/templates/books/book_confirm_delete.html #}  
<form method="post">{% csrf_token %}  
  Delete "{{ object }}" ? <input type="submit" value="Yes, delete">  
</form>
```

 REMEMBER

Compare with 2.2: the ONLY structural difference between OneToOne and ForeignKey without forms is the dropdown filter — FK offers ALL owners every time; OneToOne had to hide already-linked books.

2.4 · ManyToMany CRUD — no forms.py

Scenario: Book ↔ Categories with hand-written checkboxes and `request.POST.getlist()` .

models.py

```
# books/models.py (identical to Chapter 1.4)
from django.db import models

class Category(models.Model):
    name = models.CharField(max_length=50)

    def __str__(self):
        return self.name

class Book(models.Model):
    name = models.CharField(max_length=200)
    pages = models.IntegerField()
    categories = models.ManyToManyField(Category, related_name='books')

    def __str__(self):
        return self.name
```

views.py

```
# books/views.py
from django.shortcuts import render, redirect, get_object_or_404
from .models import Book, Category

def book_list(request):
    # READ (all)
    books = Book.objects.prefetch_related('categories')
    return render(request, 'books/book_list.html', {'object_list': books})

def book_create(request):
    # CREATE
    if request.method == 'POST':
        book = Book.objects.create(
            name = request.POST.get('name'),
            pages = request.POST.get('pages'),
        )
        # getlist -> ALL ticked checkboxes, e.g. ['1', '3']
        book.categories.set(request.POST.getlist('categories'))
        return redirect('book_list')
    categories = Category.objects.all()
    return render(request, 'books/book_form.html', {'categories': categories})

def book_update(request, pk):
    # UPDATE
    book = get_object_or_404(Book, pk=pk)
    if request.method == 'POST':
        book.name = request.POST.get('name')
        book.pages = request.POST.get('pages')
        book.save()
        book.categories.set(request.POST.getlist('categories')) # replace links
        return redirect('book_list')
    categories = Category.objects.all()
    return render(request, 'books/book_form.html',
        {'categories': categories, 'book': book})

def book_delete(request, pk):
    # DELETE
    book = get_object_or_404(Book, pk=pk)
```

```

if request.method == 'POST':
    book.delete()
    return redirect('book_list')
return render(request, 'books/book_confirm_delete.html', {'object': book})

```

urls.py

```

# books/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('', views.book_list, name='book_list'),
    path('new/', views.book_create, name='book_new'),
    path('<int:pk>/edit/', views.book_update, name='book_edit'),
    path('<int:pk>/delete/', views.book_delete, name='book_delete'),
]

```

book_form.html

```

{# books/templates/books/book_form.html - manual checkboxes #}
<h1>Book Form</h1>
<form method="post">
    {% csrf_token %}
    <p>Name:
        <input type="text" name="name"
            value="{{ book.name|default_if_none:'' }}"></p>
    <p>Pages:
        <input type="number" name="pages"
            value="{{ book.pages|default_if_none:'' }}"></p>
    <p>Categories:</p>
    {% for c in categories %}
        <label>
            <input type="checkbox" name="categories" value="{{ c.pk }}"
                {% if book and c in book.categories.all %}checked{% endif %}>
                {{ c.name }}
        </label><br>
    {% endfor %}
    <input type="submit" value="Save">
</form>

```

book_list.html · book_confirm_delete.html

```

{# books/templates/books/book_list.html #}
<h1>Books</h1>
<table border="1">
    <tr><th>Name</th><th>Categories</th><th colspan="2">Actions</th></tr>
    {% for book in object_list %}
    <tr>
        <td>{{ book.name }}</td>
        <td>
            {% for cat in book.categories.all %}
                {{ cat.name }}{% if not forloop.last %}, {% endif %}
            {% empty %} (none) {% endfor %}
        </td>
        <td><a href="{% url 'book_edit' book.pk %}">edit</a></td>
        <td><a href="{% url 'book_delete' book.pk %}">delete</a></td>
    </tr>
    {% endfor %}
</table>

```

```
<a href="{% url 'book_new' %}">New book</a>

{% books/templates/books/book_confirm_delete.html %}
<form method="post">{% csrf_token %}
  Delete "{{ object }}" ? <input type="submit" value="Yes, delete">
</form>
```

⚠ WATCH OUT

Three keys for manual M2M: (1) save the Book FIRST — it needs a pk before links can exist; (2) `getlist()` not `get()` — checkboxes send multiple values under one name; (3) `set()` replaces the whole link set, which is exactly what an edit form needs.

CHAPTER 3

Sharing one HTML page for Create + Update

You already saw that create and update render the SAME template. This chapter explains WHY it works and shows the cleanest patterns — with `forms.py` and without — using the Owner/Book scenario.

3.1 · Why sharing works (with forms.py)

A `ModelForm` renders whatever is inside it. An **unbound** form renders empty inputs; a form built with `instance=book` renders the book's current values. The template just prints `{{ form.as_p }}` — it never needs to know which case it is.

views.py – two views, one template

```
# books/views.py – create and update pointing at ONE template
def book_create(request):
    form = BookForm(request.POST or None)          # unbound -> empty inputs
    if form.is_valid():
        form.save()
        return redirect('book_list')
    return render(request, 'books/book_form.html',
                  {'form': form, 'title': 'Add a new book'})

def book_update(request, pk):
    book = get_object_or_404(Book, pk=pk)
    form = BookForm(request.POST or None, instance=book) # bound -> pre-filled
    if form.is_valid():
        form.save()
        return redirect('book_list')
    return render(request, 'books/book_form.html',
                  {'form': form, 'title': f'Edit {book.name}'})
```

book_form.html (shared)

```
{# books/templates/books/book_form.html – ONE file for both #}
<h1>{{ title }}</h1>                                {# 'Add a new book' or 'Edit Django Basics' #}
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <input type="submit" value="Save">
</form>
```

★ EXAM TIP

The `title` context variable is the standard way to make the shared page say different things. You can pass any extra flags the same way, e.g. `{'is_edit': True}`.

3.2 · Going further: ONE view for both create and update

Since the only difference is `instance=`, you can merge the two views into one function with an optional `pk`:

views.py – combined view

```
# books/views.py – ONE view + ONE template for create AND update
def book_form(request, pk=None):
    book = get_object_or_404(Book, pk=pk) if pk else None
    form = BookForm(request.POST or None, instance=book) # instance=None is OK!
    if form.is_valid():
        form.save()
        return redirect('book_list')
    title = f'Edit {book.name}' if book else 'Add a new book'
    return render(request, 'books/book_form.html',
                  {'form': form, 'title': title})
```

urls.py

```
# books/urls.py — two URLs, same view
urlpatterns = [
    path('new/', views.book_form, name='book_new'),      # pk=None
    path('<int:pk>/edit/', views.book_form, name='book_edit'), # pk given
]
```

★ EXAM TIP

`instance=None` is perfectly valid — the `ModelForm` just behaves as a create form. That's the whole trick behind the combined view. Great exam answer for 'how do you avoid duplicate code between create and update?'

3.3 · The same idea for every relationship (with forms.py)

Nothing changes! Because `{{ form.as_p }}` renders dropdowns (OneToOne / ForeignKey) and checkboxes (ManyToMany) automatically, the SAME shared template works for all four cases. Only the form class differs:

Case	Form	Shared template renders
Normal	<code>BookForm(name, pages)</code>	2 text inputs
OneToOne	<code>BookProfileForm(book, isbn, summary)</code>	dropdown + inputs; pre-selected on edit
One-to-Many	<code>BookForm(owner, name, pages)</code>	owner dropdown; pre-selected on edit
ManyToMany	<code>BookForm(name, pages, categories)</code>	checkboxes; pre-ticked on edit

3.4 · Shared template WITHOUT forms.py

Manual inputs can also be shared — the pattern is: **always render from an object that may be missing**. On create the view sends no object, so every value collapses to empty/unchecked; on update the object fills everything. Scenario: an **Owner has many Books**.

views.py – combined view, manual style

```
# books/views.py – no forms.py, one combined view, one shared template
from django.shortcuts import render, redirect, get_object_or_404
from .models import Book, Owner

def book_form(request, pk=None):
    book = get_object_or_404(Book, pk=pk) if pk else None

    if request.method == 'POST':
        if book is None:                # CREATE branch
            book = Book()
            book.owner_id = request.POST.get('owner')    # UPDATE fields
            book.name     = request.POST.get('name')
            book.pages    = request.POST.get('pages')
            book.save()
            return redirect('book_list')

    return render(request, 'books/book_form.html', {
        'book':    book,                # None on create
        'owners':  Owner.objects.all(),
        'title':   f'Edit {book.name}' if book else 'Add a new book',
    })
```

book_form.html (shared, manual)

```
{# books/templates/books/book_form.html – shared, manual style #}
<h1>{{ title }}</h1>
<form method="post">
    {% csrf_token %}
    <p>Owner:
        <select name="owner">
            {% for o in owners %}
            <option value="{{ o.pk }}"
                {% if book.owner_id == o.pk %}selected{% endif %}>
                {{ o.name }}
            </option>
            {% endfor %}
        </select></p>

    {# book is None on create -> default_if_none gives '' -> empty input #}
    <p>Name:
        <input type="text" name="name"
            value="{{ book.name|default_if_none:'' }}"></p>
    <p>Pages:
        <input type="number" name="pages"
            value="{{ book.pages|default_if_none:'' }}"></p>
    <input type="submit" value="Save">
</form>
```

REMEMBER

Why it works on create: `{{ book.name }}` with `book=None` renders as empty (Django templates fail silently), and `book.owner_id == o.pk` is never True, so no option is pre-selected. The same file behaves as a blank form.

3.5 · Shared manual template for ManyToMany

M2M checkbox fragment (shared)

```
{# fragment for a shared manual M2M form – same collapse trick #}
{% for c in categories %}
    <label>
        <input type="checkbox" name="categories" value="{{ c.pk }}"
            {% if book and c in book.categories.all %}checked{% endif %}>
        {{ c.name }}
    </label><br>
{% endfor %}

{# view side (POST branch): #}
{#     book.save()                                     #}
{#     book.categories.set(request.POST.getlist('categories')) #}
```

3.6 · Summary — the two sharing patterns

	With forms.py	Without forms.py
What is shared	Template prints <code>{{ form.as_p }}</code>	Template renders from an object that may be None
Create vs update switch	<code>instance=book</code> or nothing	Pass <code>book</code> or don't
Pre-filling	Automatic (bound form)	Manual: <code>value=""</code> , <code>selected</code> , <code>checked</code> conditions
Combined view trick	<code>def book_form(request, pk=None) + instance</code>	Same signature + <code>Book()</code> when <code>pk</code> is None
Validation	Free (<code>is_valid</code> , error messages)	You must write it yourself

★ EXAM TIP

Final revision: `forms.py` = less template code, free validation, automatic pre-fill. Manual = full control of the HTML but you re-implement everything. Exams often ask exactly this comparison — now you have written both versions of all four CRUDs, so you can answer with code.